

이벤트 시스템 — JSON 스키마 안 (A · B · C)

선행 안: 이벤트 시스템 구현 - 안 3.md

목적: 이벤트 한 단위를 JSON 로드 가능한 dict 로 표현할 때, 자료구조 스키마를 어떻게 가져갈지 세 갈래로 비교.
안 3 의 흐름·매니저 분리·시그널 계약은 그대로 두고 데이터 모양만 바꾸는 결정.

§0. 공통 가정 (세 안 모두 동일)

- 이벤트 = JSON dict 한 개 = 상태 정의자. 외부 .json 또는 const dict 둘 다 가능. 처음엔 const, 임계점 후 외부 파일.
- on/off 배타 lifecycle. 이벤트 활성화 중 외부 입력·다른 매니저 상태 변경 무시. EventManager.event_state 살아있는 동안 RunManager / run_ui 외 입력 lockout.
- Type 1 (mutating) = RunManager._apply_* 직 호출 단일 경로. escape 없음.
- Ren'Py 결 어휘: say / show / scene / menu / play_movie 등 — 단 label / jump / goto 같은 흐름 제어는 미도입. 분기는 데이터 트리(또는 영역 분리) 로만.
- MVP = 대사 (say) 만 + Type 1 효과. 배경·캐릭터·선택지·동영상은 추후. 다만 스키마는 처음부터 그 자리를 뒀.
- JSON 직렬화 호환: Color() 등 GDScript-only 타입 안 씀. 색·경로 모두 문자열.

§1. 안 A — 평면 액션 시퀀스 (Ren'Py imperative 결)

MVP 스키마

```
{
  "id": "intro_monologue",
  "trigger": {"type": "node_enter", "node_type": "event"},
  "once_per": "big_run",
  "actions": [
    {"type": "say", "speaker": "heroine", "text": "여긴 어디지..."},
    {"type": "say", "speaker": "heroine", "text": "팔이 움직이지 않아."},
    {"type": "body_boost", "amount": -5}
  ]
}
```

- 하나의 평면 액션 배열. Type 1·Type 2 가 같은 시퀀스에 자유 섞임.

- 진행: cursor 가 actions 순차 소비. say 만나면 UI 로 라인 통보 후 사용자 advance 대기. Type 1 은 즉시 실행 + 다음으로.

추후 확장

배경·캐릭터·선택지·동영상 = 모두 액션 타입 추가:

```
"actions": [
  {"type": "scene", "background": "bg_lab"},
  {"type": "show", "side": "L", "char_id": "heroine", "emotion": "hurt"},
  {"type": "say", "speaker": "heroine", "text": "..."},
  {"type": "menu", "choices": [
    {"label": "응급 처치", "actions": [
      {"type": "say", "speaker": "heroine", "text": "..."},
      {"type": "body_boost", "amount": -5}
    ]},
    {"label": "포기", "actions": [
      {"type": "say", "speaker": "heroine", "text": "..."}
    ]}
  ]},
  {"type": "play_movie", "path": "res://cutscenes/intro.ogv"}
]
```

선택지는 액션의 한 종류 (menu). 그 가지를 고르면 가지의 actions 가 cursor 의 자식 시퀀스로 진입.

장단

강점	최대 유연성. Type 1·2 자유 인터리빙. 라인 사이에 효과 끼우기 자연스러움. Ren'Py 의 imperative 스크립트와 거의 1:1 — 어휘 추가가 곧 액션 type 추가.
약점	데이터 검증 약함 — 모든 게 한 배열이라 "이 이벤트는 라인 몇 개?" 같은 질문이 순회로만 답 가능. 작가가 잘못된 액션 type 적어도 런타임 전엔 모름. 표/스프레드시트 친화도 낮음.

§2. 안 B — 영역 분리 (lines / effects / choices)

MVP 스키마

```
{
  "id": "intro_monologue",
  "trigger": {"type": "node_enter", "node_type": "event"},
  "once_per": "big_run",
  "lines": [
    {"speaker": "heroine", "text": "여긴 어디지..."},
    {"speaker": "heroine", "text": "팔이 움직이지 않아."}
  ]
}
```

```

],
"effects": [
  {"type": "body_boost", "amount": -5}
]
}

```

- 라인은 라인끼리, 효과는 효과끼리 별도 영역.
- 진행: 모든 라인 순차 → 끝나면 effects 일괄 적용 → `event_resolved`.
- 효과 적용 시점은 약속으로 "라인 후" 고정.

추후 확장

각 영역이 더 늘어남:

```

{
  "id": "...",
  "scene": {"background": "bg_lab"},
  "characters": [{"side": "L", "char_id": "heroine", "emotion": "default"}],
  "lines": [
    {"speaker": "heroine", "text": "...", "emotion": "hurt"}
  ],
  "choices": [
    {
      "label": "응급 처치",
      "lines": [{"speaker": "heroine", "text": "..."}],
      "effects": [{"type": "body_boost", "amount": -5}]
    },
    {
      "label": "포기",
      "lines": [{"speaker": "heroine", "text": "..."}]
    }
  ],
  "cutscene": {"path": "..."},
  "effects": []
}

```

각 영역은 명확한 의미와 시점을 가짐. `choices` 가 있으면 `effects` 는 가지에서, 없으면 최상위 `effects` 일괄.

장단

강점	명세 자체가 자료구조에 박힘. 작가가 "이 이벤트의 모든 라인" / "모든 효과" 한눈에 봄. JSON 스키마 검증 쉬움 (필드 타입·필수성 명확). 안 3 의 Type 1·2 분류가 데이터 모양에도 그대로 반영.
약점	라인 사이에 효과 끼우기 어색 (효과는 항상 일괄). "라인 3 출력 후 HP -5, 라인 4 출력 후 HP -3" 같은 시퀀싱 표현 불가능 — 이벤트를 둘로 쪼개야 함. Ren'Py 어휘와 1:1 매핑 안 됨 (어휘는 액션, 영역은 카테고리).

§3. 안 C — 라인별 상태 부착 (frame-style)

MVP 스키마

```
{
  "id": "intro_monologue",
  "trigger": {"type": "node_enter", "node_type": "event"},
  "once_per": "big_run",
  "lines": [
    {
      "speaker": "heroine",
      "text": "여긴 어디지...",
      "effects": []
    },
    {
      "speaker": "heroine",
      "text": "팔이 움직이지 않아.",
      "effects": [{"type": "body_boost", "amount": -5}]
    }
  ]
}
```

- 라인 entry 가 곧 한 "프레임" — 자기 완결적 단위. 라인 텍스트 + 그 시점의 효과를 함께 포함.
- 진행: 라인 N 표시 → 사용자 advance 시 N 의 effects 일괄 실행 → 라인 N+1 로.

추후 확장

라인 entry 가 비대해지면서 한 라인이 곧 한 화면 상태:

```
"lines": [
  {
    "speaker": "heroine",
    "text": "...",
    "background": "bg_lab",
    "characters": {"L": {"id": "heroine", "emotion": "hurt"}},
    "effects": [],
    "choices": null
  },
  {
    "speaker": null,
    "text": null,
    "background": "bg_lab",
    "characters": {},
    "cutscene": {"path": "..."},
    "effects": []
  },
]
```

```

{
  "speaker": "heroine",
  "text": "어떻게 할까?",
  "characters": {"L": {"id": "heroine", "emotion": "default"}},
  "choices": [
    {"label": "응급 처치", "lines": [...]},
    {"label": "포기", "lines": [...]}
  ]
}
]

```

각 라인이 자기 시점의 화면 전체를 명시 — 작가가 표 1행 단위로 작성.

장단

강점	라인 단위 효과·상태 자연스러움. 작가가 "이 라인에서 뭐가 보이지?" 한 줄로 답. 표/스프레드시트 1:1 매핑 — 작가 친화 최고. 누적 상태 추적 부담 0 (각 라인 자기 완결).
약점	같은 배경/캐릭터가 라인 N개 동안 안 변해도 매번 명시 — 노이즈. 라인 사이 분기는 choices 의 lines nested 로만 표현 — 라인 entry 가 라인 시퀀스를 다시 품는 재귀 구조. 라인 entry 비대화.

§4. 비교 표

측면	A (평면)	B (영역 분리)	C (라인-프레임)
Type 1:2 인터리빙	✅ 자유	❌ 어려움	✅ 라인 단위
Ren'Py 어휘 1:1	✅ 매우 가까움	❌ 카테고리 vs 어휘	△ 부분적
작가 직관 (한눈에)	△ 순회 필요	✅ 영역별 일목	✅ 표 1행
스키마 검증	△ 약함	✅ 강함	✅ 강함 (라인 단위)
누적 상태 추적	△ 필요	△ 영역별	✅ 불필요
데이터 노이즈	✅ 낮음	△ 중간	❌ 높음 (반복)
MVP 단순성	✅ 단순	✅ 가장 단순	△ 약간 비대
확장 시 변형성	✅ 액션 추가만	△ 영역 추가	△ 라인 필드 추가
EventManager 구현비	△ cursor + nested (menu)	✅ 낮음	✅ 낮음

§5. 결정 포인트

세 안의 갈림은 결국 "라인과 효과의 관계"에 대한 입장:

- **A** — 라인도 효과도 같은 시퀀스 안의 동급 시민. Ren'Py 그대로.
- **B** — 라인은 라인, 효과는 효과. 분류 명확, 인터리빙 포기.
- **C** — 라인이 효과를 품음. 라인 = 한 화면 = 한 상태.

확정에 필요한 4가지 질문:

1. **이벤트 도중 효과 시점을 쪼갤 일이 있는가?** - Yes → A 또는 C. (B는 일괄만 가능) - No → 셋 다 가능 → B가 가장 가벼움.
2. **작가 작성이 표/스프레드시트 친화여야 하는가?** - Yes → C가 압도, B가 차선. A는 한 행이 평면 배열이라 작성 까다로움. - No → 셋 다 무방.
3. **추후 어휘가 풍부해지는가, 영역이 풍부해지는가?** - 어휘 (`scene` / `show` / `play_movie` 등) 풍부 → A. - 영역 (배경 영역, 캐릭터 영역, 선택지 영역...) 풍부 → B. - 라인 필드 풍부 → C.
4. **MVP 가벼움 vs. 확장 자연스러움 어디 무게?** - MVP 가벼움 = B가 가장 적은 줄로 시작. - 확장 자연스러움 = A (액션 풀 추가) 또는 C (라인 필드 추가).

§6. 한줄 평

- **B의 매력:** MVP 단계에서 자료구조가 가장 명료. 빠른 진입.
- **A의 매력:** 추후 확장 시 Ren'Py 어휘 풀 그대로 흡수. 인터리빙 자유.
- **C의 매력:** 작가 친화 + 라인 단위 효과. 단 노이즈 감수 필요.

확정 결정자 한 질문: "이벤트 도중 효과 시점 쪼개기가 필요한가?" Yes면 B 탈락, A·C 중 선택.